

CHAPITRE 1

DEBOGUER LE CODE PYTHON

1- Gestion des erreurs (compilation-syntaxe)

2-Débogage

3-Outils de suivi et de visualisation de
l'exécution d'un code python



01- DEBOGUEUR LE CODE PYTHON

Gestion des erreurs (compilation-syntaxe)



Erreurs de syntaxe

- Les erreurs de syntaxe sont des erreurs d'analyse du code

Exemple:

```
>>> while True print('Hello world')  
SyntaxError: invalid syntax
```

- L'analyseur indique la ligne incriminée et affiche une petite « flèche » pointant vers le premier endroit de la ligne où l'erreur a été détectée.
- L'erreur est causée par le symbole placé avant la flèche.
- Dans cet exemple, la flèche est sur la fonction print() car il manque deux points (':') juste avant. Le nom du fichier et le numéro de ligne sont affichés pour vous permettre de localiser facilement l'erreur lorsque le code provient d'un script.

01- DEBOGUER LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)



Exceptions

- Même si une instruction ou une expression est syntaxiquement correcte, elle peut générer une erreur lors de son exécution.
- Les erreurs détectées durant l'exécution sont appelées des **exceptions** et ne sont pas toujours fatales
- La plupart des exceptions toutefois ne sont pas prises en charge par les programmes, ce qui génère des messages d'erreurs comme celui-ci :

```
>>> 10 * (1/0)

Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    10 * (1/0)
ZeroDivisionError: division by zero
```

- La dernière ligne du message d'erreur indique ce qui s'est passé. Les exceptions peuvent être de différents types et ce type est indiqué dans le message : le type indiqué dans l'exemple est **ZeroDivisionError**

01- DEBOGUEUR LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)



Instruction try-except

- Python présente un **mécanisme d'exception** qui permet de gérer des exécutions exceptionnelles qui ne se produisent qu'en cas d'erreur.
- La construction de base à utiliser est l'instruction **try-except** qui se compose de deux blocs de code.
- On place le code « risqué » dans le bloc try et le code à exécuter en cas d'erreur dans le bloc except.
- **Exemple:**
- On souhaite calculer l'âge saisi par l'utilisateur en soustrayant son année de naissance à 2016. Pour cela, il faut convertir la valeur de la variable birthyear en un int. Cette conversion peut échouer si la chaîne de caractères entrée par l'utilisateur n'est pas un nombre.

```
birthyear = input('Année de naissance ? ')

try:
    print('Tu as', 2016 - int(birthyear), 'ans.')
except:
    print('Erreur, veuillez entrer un nombre.')

print('Fin du programme.')
```

01- DEBOGUER LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)



Instruction try-except

```
birthyear = input('Année de naissance ? ')
try:
    print('Tu as', 2016 - int(birthyear), 'ans.')
except:
    print('Erreur, veuillez entrer un nombre.')
print('Fin du programme.')
```

1er cas

- Si l'utilisateur entre un nombre entier, l'exécution se passe sans erreur et son âge est calculé et affiché

Année de naissance ? 1994
Tu as 22 ans.
Fin du programme.

2ème cas

- Si l'utilisateur entre une chaîne de caractères quelconque, qui ne représente pas un nombre entier, un message d'erreur est affiché

Année de naissance ? deux
Erreur, veuillez entrer un nombre.
Fin du programme.

- Dans le premier cas, la conversion s'est passée normalement, et le bloc try a donc pu s'exécuter intégralement sans erreur.
- Dans le second cas, une erreur se produit dans le bloc try, lors de la conversion. L'exécution de ce bloc s'arrête donc immédiatement et passe au bloc except, avant de continuer également après l'instruction try-except.

01- DEBOGUEUR LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)

Type Exception

- Lorsqu'on utilise l'instruction **try-except**, le bloc **except** capture toutes les erreurs possibles qui peuvent survenir dans le bloc try correspondant.
- Une **exception** est en fait représentée par un objet, instance de la classe Exception.
- On peut récupérer cet objet en précisant un nom de variable après **except**

```
try:
    a = int(input('a ? '))
    b = int(input('b ? '))
    print(a, '/', b, '=', a / b)
except Exception as e:
    print(type(e))
    print(e)
```

- On récupère donc l'objet de type Exception dans la variable e.
- Dans le bloc except, on affiche son type et sa valeur.

01- DEBOGUEUR LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)

Type Exception

Voici deux exemples d'exécution qui révèlent deux types d'erreurs différents :

- Si on ne fournit pas un nombre entier, il ne pourra être converti en **int** et une erreur de type **ValueError** se produit :

```
>>> try:
    a = int(input('a ? '))
    b = int(input('b ? '))
    print(a, '/', b, '=', a / b)
except Exception as e:
    print(type(e))
    print(e)

a ? trois
<class 'ValueError'>
invalid literal for int() with base 10: 'trois'
```

- Si on fournit une valeur de 00 pour b, on aura une division par zéro qui produit une erreur de type **ZeroDivisionError** :

```
>>> try:
    a = int(input('a ? '))
    b = int(input('b ? '))
    print(a, '/', b, '=', a / b)
except Exception as e:
    print(type(e))
    print(e)

a ? 5
b ? 0
<class 'ZeroDivisionError'>
division by zero
```

```
try:
    a = int(input('a ? '))
    b = int(input('b ? '))
    print(a, '/', b, '=', a / b)
except Exception as e:
    print(type(e))
    print(e)
```

01- DEBOGUEUR LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)



Capture d'erreur spécifique

- Chaque type d'erreur est donc défini par une classe spécifique.
- Il est possible d'associer plusieurs blocs except à un même bloc try, pour exécuter un code différent en fonction de l'erreur capturée.
- Lorsqu'une erreur se produit, les blocs except sont parcourus l'un après l'autre, du premier au dernier, jusqu'à en trouver un qui corresponde à l'erreur capturée.

Exemple:

```
>>> try:
    a = int(input('a ? '))
    b = int(input('b ? '))
    print(a, '/', b, '=', a / b)
except ValueError:
    print('Erreur de conversion.')
except ZeroDivisionError:
    print('Division par zéro.')
except:
    print('Autre erreur.')
```

- Lorsqu'une erreur se produit dans le bloc **try** l'un des blocs **except** seulement qui sera exécuté, selon le type de l'erreur qui s'est produite.
- Le dernier bloc **except** est là pour prendre toutes les autres erreurs.
- Lorsqu'une erreur se produit dans le bloc **try** l'un des blocs **except** seulement qui sera exécuté, selon le type de l'erreur qui s'est produite.
- Le dernier bloc **except** est là pour prendre toutes les autres erreurs.
- **L'ordre des blocs except est très important et il faut les classer du plus spécifique au plus général, celui par défaut devant venir en dernier.**

01- DEBOGUER LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)

Gestionnaire d'erreur partagé

- Il est possible d'exécuter le même code pour différents types d'erreur, en les listant dans un tuple après le mot réservé **except**.
- Si on souhaite exécuter le même code pour une erreur de conversion et de division par zéro, il faudrait écrire :

```
try:
    a = int(input('a ? '))
    b = int(input('b ? '))
    print(a, '/', b, '=', a / b)
except (ValueError, ZeroDivisionError) as e:
    print('Erreur de calcul :', e)
except:
    print('Autre erreur.')
```

01- DEBOGUEUR LE CODE PTHON

Gestion des erreurs (compilation-syntaxe)

Bloc finally

- le mot réservé **finally** permet d'introduire un bloc qui sera exécuté soit après que le bloc try se soit exécuté complètement sans erreur, soit après avoir exécuté le bloc except correspondant à l'erreur qui s'est produite lors de l'exécution du bloc try.
- On obtient ainsi une instruction **try-except-finally**

```
print('Début du calcul.')
try:
    a = int(input('a ? '))
    b = int(input('b ? '))
    print('Résultat :', a / b)
except:
    print('Erreur.')
finally:
    print('Nettoyage de la mémoire.')
print('Fin du calcul.')
```

- Si l'utilisateur fournit des valeurs correctes pour a et b l'affichage est le suivant:

```
Début du calcul.
a ? 2
b ? 8
Résultat : 0.25
Nettoyage de la mémoire.
Fin du calcul.
```

- si une erreur se produit l'affichage est le suivant:

```
Début du calcul.
a ? 2
b ? 0
Erreur.
Nettoyage de la mémoire.
Fin du calcul.
```

Dans les 2 cas le
bloc finally a été
exécuté